

BrepCoder: 통합 멀티모달 대규모 애플리케이션 다중 작업 B-rep 추론을 위한 언어 모델

김민기, 김용준, 강정우, 김형기

충남대학교, 대한민국 대전 mingi@o.cnu.ac.kr, kimitoinf@icloud.com
mercurii713@gmail.com, hk.kim@cnu.ac.kr

요약. 최근 딥러닝 분야의 발전은 컴퓨터 지원 설계(CAD) 분야의 복잡한 문제들을 해결하는 데 크게 기여해 왔습니다. 그러나 기존의 대부분의 접근 방식은 새로운 작업을 위해 구조적 수정이 필요한 작업별 모델에 의존하며, 주로 산업 표준인 경계 표현(B-rep) 형식보다는 포인트 클라우드나 이미지에 초점을 맞추고 있습니다. 이러한 한계를 해결하기 위해, 본 논문에서는 B-rep 입력을 기반으로 다양한 CAD 작업을 수행하는 통합 멀티모달 대규모 언어 모델(MLLM)인 BrepCoder를 제안합니다. 대규모 언어 모델(LLM)의 코드 생성 기능을 활용하여 CAD 모델링 시퀀스를 파이썬과 유사한 코드로 변환하고 이를 B-rep 형식에 맞춥니다. 그런 다음 2단계 학습 전략을 채택합니다. 첫 번째 단계에서는 역공학을 통해 기하학적 특징과 설계 논리를 학습하는 사전 학습을 수행합니다. 두 번째 단계에서는 모델을 확장하여 완성, 오류 수정, CAD 품질 보증(CAD-QA)과 같은 다양한 후속 작업을 수행합니다. 결과적으로, BrepCoder는 B-rep을 구조적 코드로 해석함으로써 다양한 작업에서 뛰어난 일반화 성능을 달성하며, 범용 CAD 에이전트로서의 잠재력을 보여줍니다.

핵심어: 컴퓨터 지원 설계 · 경계 표현 · 멀티모달 대규모 언어 모델

1. 서론

컴퓨터 지원 설계(CAD)는 현대 제조 및 엔지니어링에서 기하학적 형상을 정의하고 제품 프로토타입을 설계하는 데 필수적인 기본 기술입니다. 설계 효율성을 극대화하고 지능형 자동화를 구축하려는 요구가 증가함에 따라 최근 연구에서는 텍스트, 이미지, 포인트 클라우드, 경계 표현(B-rep)과 같은 다중 모달 입력을 사용하여 복잡한 CAD 문제를 해결하기 위해 딥러닝 기술을 적극적으로 활용하고 있습니다[7, 21, 43, 48]. 특히, 설계 이력을 복구하기 위한 역설계[20, 45], 불완전한 모델링 시퀀스 완성[19], 자동화 등과 같은 중요한 산업적 과제를 해결하기 위해 광범위한 노력이 기울여지고 있습니다.

2 M. Kim 외

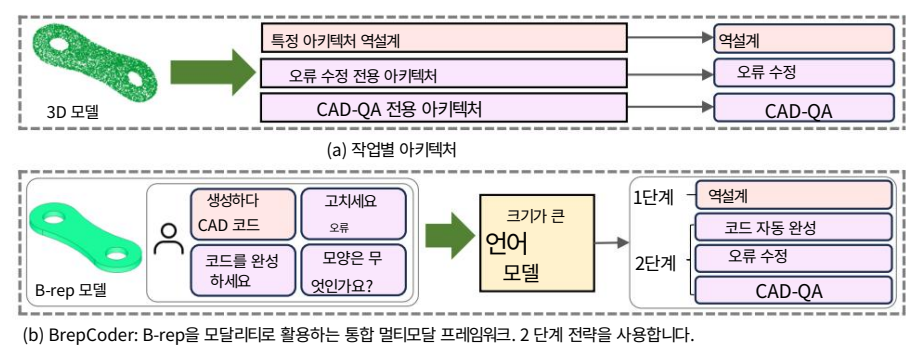


그림 1: CAD 학습 프레임워크 비교. (a) 작업별 아키텍처: 작업에 따라 다른 아키텍처를 채택하는 모델. (b) BrepCoder: 2단계 학습 전략을 사용하는 통합 MLLM 프레임워크.

오류 수정[6] 및 3D 모델에서 설계 의도의 의미론적 이해를 용이하게 하기 위한 질문 답변.

이러한 발전에도 불구하고, CAD 분야의 기존 딥러닝 모델의 대부분은 그림 1(a)에서 볼 수 있듯이 개별 작업을 위해 설계된 작업별 아키텍처에 국한되어 있습니다[12]. 이러한 접근 방식은 새로운 작업이나 설계 환경이 도입 될 때마다 구조적 수정이 필요하기 때문에 확장성 측면에서 비효율적입니다. 결과적으로, 이러한 작업별 모델은 설계 프로세스 전반에 걸쳐 서로 다른 작업 간에 공통 지식을 공유하지 못하고, 실제 산업 응용 분야의 복잡한 요구 사항을 유연하게 처리하는 데 필요한 일반화 능력이 부족합니다.

다중 모달 대규모 언어 모델(MLLM)의 등장은 이러한 과제를 해결할 수 있는 유망한 길을 제시합니다. 대규모 언어 모델(LLM)의 광범위한 사전 학습 된 지식을 활용하는 MLLM은 다재다능한 추론 및 일반화 능력을 입증해 왔습니다[1-3, 47]. MLLM은 이미지, 포인트 클라우드, B-rep과 같은 다양한 모달리티를 자연어 명령과 함께 통합하고 처리할 수 있습니다[41]. 결과적으로, LLM의 범용 추론 능력과 모달리티별 정보를 결합함으로써 모델은 CAD 형상과 설계 논리 간의 상관관계를 학습할 수 있습니다[11, 23]. 이러한 시너지 효과는 단일 MLLM 아키텍처 내에서 광범위한 CAD 작업을 수행할 수 있는 통합 프레임워크 구축을 용이하게 합니다.

모달리티 선택 관점에서 볼 때, CAD 분야의 전통적인 딥러닝 연구는 주로 이미지나 포인트 클라우드와 같은 표현 방식에 초점을 맞춰왔습니다[5, 22, 46]. 최근 연구에서는 CAD 시퀀스 복구[20, 45] 또는 MLLM 입력[9]으로 B-rep을 탐색하기 시작했지만, 이미지나 포인트 클라우드 기반 방법론에 비해 그 범위는 여전히 제한적입니다. 그러나 대다수의 산업용 CAD 모델이 B-rep 형식으로 저장되고 공유된다는 점을 고려할 때, 이를 직접 활용하는 방법론을 개발하는 것은 매우 중요합니다.

제목이 너무 길어서 생략되었습니다.

3

실질적으로 매우 중요한 의미를 지닌다. 이러한 중요성에도 불구하고, 범용 MLLM은 B-rep 데이터를 사용하여 다양한 CAD 작업을 종합적으로 처리하도록 설계되었지만, 여전히 개발이 진행 중이다. 부족한 부분이 있습니다. 이러한 격차를 해소하기 위해, 우리는 B-rep을 채택한 통합 프레임워크를 제안합니다. CAD 영역 내의 다양한 작업을 해결하기 위한 핵심 입력 방식으로.

B-rep을 LLM과 통합하려는 시도는 있었지만, 기존의 B-rep 기반 MLLM은 주로 기하학적 형태를 기술하는 작업에 국한되어 왔다.

B-rep을 자연어 캡션과 정렬하여 특성을 파악하거나 모양 범주를 식별할 수 있습니다[9]. 이러한 접근 방식은 모델이 외부 기하학적 특징을 파악하는 데 도움이 될 수 있지만, 최종 출력에 대한 이해만을 제공할 뿐입니다.

기하학적 측면에서, 구성에 필요한 구체적인 절차 단계를 포착하지 못하는 것입니다.

결과적으로, 기하학과 설계 논리에 대한 구체적인 이해 없이는 이러한 모델은 다양한 CAD 작업을 실행하는 데 근본적인 한계에 직면하게 됩니다.

모델링 생성과 같이 높은 수준의 엔지니어링 추론이 요구되는 작업

순서 수정 또는 오류 수정.

이러한 문제들을 해결하기 위해 그림 1(b)에 나타난 BrepCoder를 제안합니다.

본 프레임워크는 B-rep 데이터를 CAD 시퀀스와 정렬하여 복잡한 작업에 대한 LLM의 일반화 기능을 활용합니다. 구체적으로, 먼저 변환을 수행합니다.

모델링 시퀀스를 Python과 유사한 CAD 코드[19]로 변환하여 LLM이 효과적인 해석을 위해 강력한 프로그래밍 추론을 활용할 수 있도록 합니다. 다음으로,

변환된 CAD 코드와 B-rep을 정렬하여 점진적 학습을 위한 2단계 훈련 전략을 구현합니다. 1단계는 역설계에 중점을 둡니다.

이 단계에서 모델은 기하학적 특징과 설계 논리 간의 기본적인 상관관계를 내재화합니다. 2단계에서는 모델이 학습된 이러한 요소들을 활용합니다.

완료, 오류 등 다양한 하위 작업을 처리하기 위한 상관관계 분석

수정 및 CAD-QA를 담당했습니다. 주요 기여 사항은 다음과 같이 요약됩니다.

- 본 논문에서는 B-rep을 핵심 입력으로 채택한 통합 프레임워크인 BrepCoder를 제안합니다.
 - 모달리티. 이러한 기하학적 표현을 파이썬과 유사한 CAD 도구와 정렬합니다.
 - 코드를 통해 LLM은 복잡한 설계 논리를 직접 해석할 수 있습니다.
- 본 논문에서는 B-reps와 CAD 코드 간의 상관관계를 학습하기 위한 2단계 학습 전략을 소개합니다. 역공학에 대한 사전 학습을 통해 다음과 같은 이점을 얻을 수 있습니다.
 - 설계 논리를 내재화하는 모델로서 강력한 일반화 능력을 확보합니다.
 - 다양한 하위 작업을 위해.
- BrepCoder는 습득한 설계 로직을 활용하여 고도의 엔지니어링이 요구되는 복잡한 CAD 영역 작업에서 더 높은 성능을 보여줍니다.
 - 역공학, 완성, 오류 수정 등을 포함한 추론
 - CAD-QA.

2. 관련 연구

3D CAD 재구성을 위한 딥러닝. 최신 3D CAD 생성

연구 결과는 설계 이력을 통합한 파라메트릭 모델링을 선호합니다. DeepCAD

[35]는 이 과정을 순차적 연산을 통해 생성된 것으로 공식화하는 데 선구적인 역할을 했습니다.

트랜스포머 기반 오토인코더[4,33]는 이후 확장된 패러다임입니다.

다양한 양식. 예를 들어, MultiCAD [22]는 대조 학습 전략을 사용하여 관계를 포착함으로써 재구성 성능을 향상시켰습니다.

포인트 클라우드와 CAD 시퀀스 간의 관계. Img2CAD [7]는 2단계 방식을 제안했습니다. GPT-4V[36]를 활용하여 전역 구조를 예측하는 접근 방식. 매개변수 추정을 위한 별도의 모듈이 있습니다. 또한 Brep2Seq [45]는 B-rep을 그래프 구조로 해석하여 Transformer 인코더-디코더를 통해 모델링 시퀀스를 예측하고, CADCL [20]은 대조 학습 프레임워크를 도입하여 B-rep과 CAD 시퀀스 임베딩을 정렬함으로써 재구성 정확도를 향상시켰습니다. 그러나 기존 방법들은 주로 경직되고 작업별 특화된 방식에 의존합니다.

개별 재구성 작업을 위해 처음부터 학습된 아키텍처.

CAD용 대규모 언어 모델. 최근 CAD 설계는 점점 더 중요해지고 있습니다.

언어학적 관점을 채택하고 코드 생성 기능을 활용했습니다.

LLMs. [44]. Text2CAD [17]는 자연어로부터 CAD 생성을 개척했습니다.

프롬프트와 CAD-Llama[19]는 CAD 시퀀스를 구조화된 매개변수 CAD 코드(SPCC)로 변환하여 텍스트를 CAD로 변환, 완성, 추가 및 삭제와 같은 다양한 작업을 수행했습니다. 자연어 교육을 넘어, 연구는

시각 정보를 통합하는 MLLM에 대한 연구가 진행 중입니다. CAD-GPT [34]

이미지 및 텍스트 입력을 통해 재구성된 설계 이력. CAD-MLLM [38]은 이력 재구성을 위해 텍스트 및 이미지와 함께 포인트 클라우드 데이터를 통합했습니다.

CADReview [6]에서는 결함이 있는 CAD를 수정하기 위한 오류 수정 작업을 제안했습니다. 이미지 입력을 사용하는 프로그램들. 최근에는 LLM을 확장하려는 노력이 나타나고 있다.

이미지와 포인트 클라우드를 넘어 B-rep 데이터에 대한 이해. BrepLLM [9]

LLM이 훈련 전략을 통해 B-reps를 해석할 수 있음을 입증했습니다.

자연어 캡션과 일치시킵니다. 하지만 그 기능은 엄격하게 제한적입니다.

분류 및 캡션 생성과 같은 설명적인 작업에 국한됩니다.

3가지 방법

그림 2는 BrepCoder 프레임워크를 보여줍니다. 1단계에서는 B-rep 표현을 정렬합니다.

해당 CAD 코드와 함께. 2단계에서는 이러한 정렬된 모달리티를 통합합니다.

LLM으로 변환하여 2단계 전략을 통해 통합 아키텍처를 학습시킵니다.

3.1 멀티모달 정렬

CAD 코드 표현. DeepCAD[35]와 같은 기존 방법은 CAD 생성에 내재된 순차적 모델링 작업을 나타냅니다.

사용자 정의 정수 토큰. 하지만 최근에는 이러한 시퀀스를 번역하는 것이 어려워졌습니다.

코드 형식으로 변환하는 것은 사전 학습된 프로그래밍을 활용하기 위해 주목받고 있습니다.

LLM에 대한 지식[6,11,19]. LLM의 추론 능력을 극대화하기 위해,

우리는 CAD-Llama [19]에서 제안한 Python과 유사한 CAD 코드 형식을 채택합니다. 그림 3 DeepCAD에서 사용되는 단순 정수 토큰 시퀀스와 본 논문에서 제안하는 코드 기반 표현을 비교합니다. 그림 3(b)에서 볼 수 있듯이, 제안된 방법은 다음 사항을 준수합니다.

객체 지향 프로그래밍 구문을 사용하여 다음과 같은 구조적 특징을 나타낸다. (1) 객체화: 각 2D 스케치는 객체로 선언되고 관리된다.

(1) 고유 변수(예: sketch_0 = []). (2) 메서드 호출: Line, Arc, Circle과 같은 기하학적 명령은 관련 메서드 호출로 실행됩니다.

해당 스케치 객체와 함께, 특정 매개변수가 전달됩니다.

제목이 너무 길어서 생략되었습니다.

5

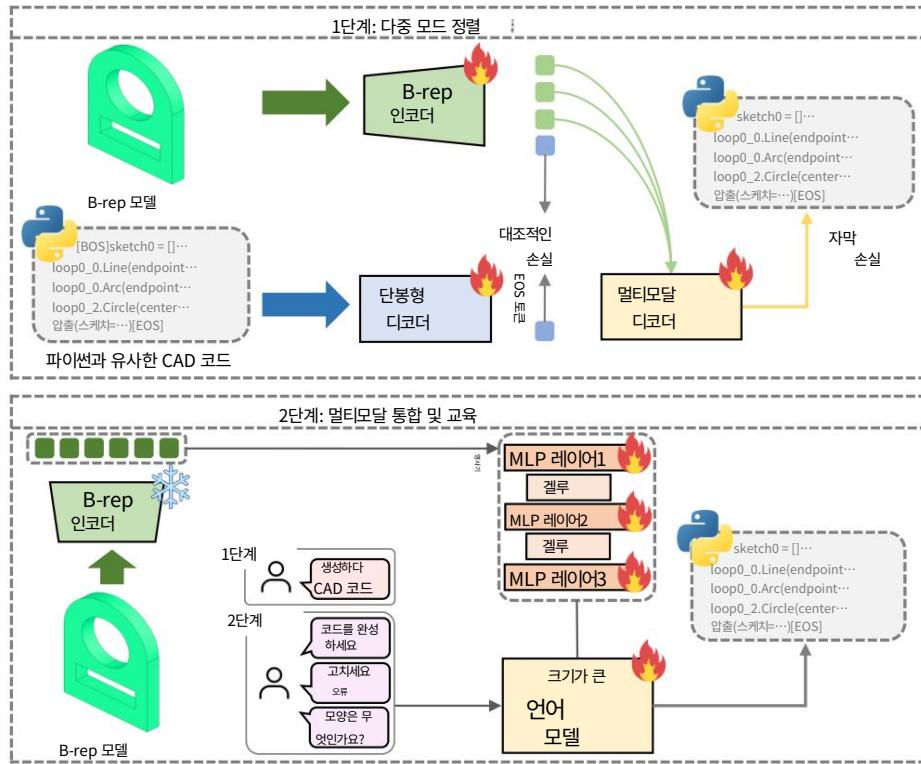


그림 2: BrepCoder의 전체 프레임워크. 1단계에서는 B-rep 표현을 CAD 코드와 정렬합니다. 2단계에서는 고정된 B-rep 인코더를 프로젝터를 통해 LLM과 통합합니다. 2단계 학습 전략을 사용하여 1단계에서는 역공학 작업을 통해 기하학적 특징과 코드 간의 대응 관계를 파악하고, 2단계에서는 다양한 후속 작업을 위해 모델을 미세 조정합니다.

인수. (3) 명시적 참조: 3D 형상을 생성하는 압출 작업은 이전에 정의된 스케치 변수를 특정 압출 매개변수와 함께 명시적 인수(예: sketch=schema_0)로 받습니다.

B-rep 특징 인코딩. 포인트 클라우드나 복셀과 같은 이산적인 3D 표현과는 달리, B-rep은 연속적인 기하학적 형태와 그 위상적 연결을 통해 3D 형상을 정의합니다. 이러한 표현을 인코딩하기 위해 BrepNet [18]과 같은 일부 방법론은 점, 곡선, 표면을 개별 노드로 처리하는 이중 그래프를 사용합니다. 그러나 이러한 접근 방식은 그래프 복잡성을 크게 증가시키고 계산 효율성을 저하시킵니다. 따라서 본 연구에서는 표면을 단일 노드로 구성하여 위상 구조를 단순화하는 UV-Net [13]을 인코더로 채택했습니다. UV-Net의 방법론에 따라 B-rep 데이터를 면 인접 그래프로 변환합니다. 여기서 노드는 표면을 나타내고 에지는 연결된 표면 간의 인접성을 나타냅니다. 표면과 곡선의 기하학적 정보는 각각 2D 및 1D 그리드에 투영됩니다.

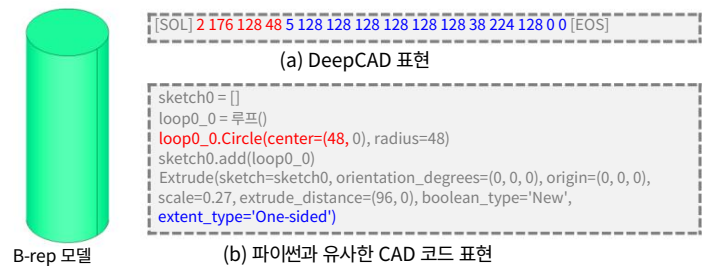


그림 3: CAD 표현 비교. (a) DeepCAD로 구성된 표현 (a) 정수 토큰. (b) 명시적으로 설명하는 파이썬과 유사한 CAD 코드 표현 설계 논리.

투영 이미지는 2D 및 1D 컨볼루션을 통해 지역 기하학적 특징으로 추출됩니다. 이러한 작업들은 이후 전체 위상 구조에 걸쳐 통합되었습니다. 그래프 신경망(GNN) [37]을 통해. 특히, B-rep 입력의 경우 N 개의 표면을 가진 UV-Net은 로컬 특징 임베딩 $E_{\text{node}} \in \mathbb{R}^{N \times 128}$ 을 생성합니다. 각 노드에 대해 그리고 다음을 통해 얻은 지역 특징 임베딩 $E_{\text{shape}} \in \mathbb{R}^{1 \times 128}$ 폴링을 수행합니다. 마지막으로, 이 두 임베딩을 연결하여 최종 B-rep을 얻습니다. 특징, $z_{\text{brep}} \in \mathbb{R}^d$. 결과적으로 z_{brep} 는 세밀한 부분을 모두 포함합니다. 각 개별 표면의 기하학적 세부 사항과 전체적인 글로벌 맥락 전체 3D 모델. CoCa 기반 멀티모달 정렬. 기존 대조 학습 기반 CLIP[28]과 같은 방법론은 지역 정렬을 수행하는 데 탁월합니다. 두 가지 양식이 있으며 검색 및 분류와 같은 작업에 효과적입니다. 따라서 대조 학습은 두 가지 양식을 구분하는 데 효과적인 접근 방식입니다. 다양한 B-rep 모델 간의 고유한 기하학적 차이와 정렬 각 유형의 고유한 특징을 해당 도형과 일대일로 대응시킵니다. CAD 코드. 하지만 이 연구의 궁극적인 목표는 분류에 국한되지 않습니다. 우리의 목표는 B-rep 피처를 기반으로 정확한 CAD 코드를 생성하는 것입니다. LLM과 통합될 때, 따라서 인코더는 명시적으로 안내되어야 합니다. 지역적인 특징뿐만 아니라 형상의 세밀한 기하학적 세부 사항까지 포착하기 위해, 우리는 CoCa [42] 프레임워크를 채택합니다. 표현 정렬을 위한 대조 손실과 캡션 손실을 통합합니다. 코드 생성을 위해.

먼저, CoCa의 계단식 주의력 폴링 메커니즘을 적용합니다. B-rep 인코더에서 얻은 특징 z_{brep} . 이 접근 방식은 처음에 다음을 추출합니다. 코드 생성에 필요한 특징들을 추출하고, 이를 요약하여 대조 학습에 필요한 특징들을 도출합니다. 구체적으로, z_{brep} 는 먼저 다음 과정을 거칩니다. 이를 $z'_{\text{brep}} \in \mathbb{R}^d$ 에 투영하기 위한 선형 레이어. 세밀한 코드 작성을 용이하게 하기 위해 세대를 정의하기 위해, 우리는 $n_{\text{query}}=256$ 개의 은닉 차원 $d=512$ 를 갖는, donly 초기화된 학습 가능한 쿼리로 구성된 $Q_{\text{gen}} \in \mathbb{R}^{256 \times 512}$ 를 정의하고, 세대 특징 z_{gen} 을 추출합니다. 마지막으로, 멀티헤드 어텐션을 활용하여

제목이 너무 길어서 생략되었습니다. 7

핵심이자 가치로서:

(1)

이어서, 대조 학습을 위해 무작위로 초기화된 학습 가능한 변수를 정의합니다.

쿼리 $Q_{con} \in \mathbb{R}^{1 \times \dots}$. 식 (1)에서 생성된 z_{gen} 을 둘 다 사용합니다.

512) ($n_{query}=1$, 키 및 값)을 사용하여 전역 특징 z_{con} 을 추출합니다. 이 특징은 다음을 캡슐화합니다.

전체 B-rep 기하학:

(2)

추출된 특징들은 유니모달 디코더와 멀티모달 디코더를 사용하여 공동으로 학습됩니다. 유니모달 디코더는 CAD 코드 입력만을 처리하며, 트랜스포머 인코더와 동일한 아키텍처를 공유합니다.

멀티모달 디코더는 트랜스포머 디코더와 동일한 아키텍처를 특징으로 합니다.

추출된 B-rep 피쳐와 상호 작용하여 CAD 코드를 생성합니다.

이 프레임워크에서 모델은 두 가지 서로 다른 손실 함수를 사용하여 최적화됩니다. 첫째, 전역 정렬을 설정하기 위해 대조 손실 함수인 $\text{sim}(\cdot, \cdot)$ 을 도입합니다.

B-rep과 CAD 코드 사이의 손실입니다. 이 손실은 Unimodal Decoder를 통해 처리된 CAD 코드의 [EOS] to-ken 임베딩 t_{eos} 를 사용하여 계산됩니다.

그리고 전역 B-rep 특징 z_{con} 은 식 (2)에서 도출됩니다. 정확한 공식은 다음과 같습니다.

다음과 같이 정의됩니다.

(3)

여기서 $\text{sim}(\cdot, \cdot)$ 은 코사인 유사도 함수를 나타내고, η 는 온도 매개변수를 나타냅니다. 다음으로, 코드 생성을 위해 캡션 손실 loss_{cap} 을 적용합니다.

이 과정에서 멀티모달 디코더는 다음과 같은 정보를 입력받아 CAD 코드를 생성합니다.

B-rep 기능 z_{gen} 을 키와 값으로 사용하고 전역 CAD 코드 기능을 사용합니다.

t_{eos} 를 쿼리로 사용합니다. 공식은 다음과 같이 정의됩니다.

(4)

여기서 t_k 는 정답 CAD 코드 토큰을 나타내고 $t_{<k}$ 는 다음을 나타냅니다.

선행 CAD 코드 토큰. 마지막으로, 모델은 가중치를 통해 최적화됩니다.

두 손실 함수의 합입니다. 총 손실은 다음과 같이 공식화됩니다.

(5)

여기서 λ_{con} 과 λ_{cap} 은 각각의 손실에 대한 가중치를 나타냅니다. 우리는 다음을 채택합니다.

CoCa에서 최적 구성은 $\lambda_{con}=1$ 및 $\lambda_{cap}=2$ 로 설정됩니다.

3.2 멀티모달 통합 및 교육

다중 모달 통합. 정렬 훈련을 통해 B-rep 인코더를 구현합니다.

CAD에 맞춰 정렬된 기하학적 특징 z_{brep} 을 추출할 수 있습니다.

코드 특징. 이러한 정렬된 특징 정보를 보존하기 위해 B-rep을 고정합니다.
인코더를 LLM과 통합하여 MLLM 아키텍처를 구축합니다.
이 과정에서 프로젝트 모듈을 삽입하여 불일치를 해결합니다.
인코더가 추출한 특징 공간과 임베딩 공간 사이의 차이
LLM. 프로젝트는 인코더 출력 $z_{\text{brep}} \in \mathbb{R}^{N \times 256}$ 을 입력으로 받습니다.
이를 LLM 공간으로 매핑하여 z_{proj} 를 얻습니다.

(6)

여기서 $\text{Projector}(\cdot)$ 는 세 개의 선형 레이어와 GELU 활성화 함수로 구성됩니다. 마지막으로 z_{proj} 는 텍스트 프롬프트 임베딩과 연결되어 입력됩니다.

법칙 섹서(LLM) 과정으로 진화합니다.

1단계: 역공학을 통한 구조적 논리 획득. 첫 번째
이 단계에서는 LLM이 역설계 기능을 습득하도록 훈련시켜 B-rep 형상을 해석하고 이를 명시적 모델링으로 변환할 수 있도록 합니다.
시퀀스를 학습시킵니다. 그런 다음 해당 파이프 스타일의 CAD를 생성하도록 모델을 훈련합니다.
 z_{proj} 와 텍스트 프롬프트 임베딩을 입력으로 사용하여 코드를 작성합니다. 손실
1단계에 대한 함수, $\text{protect}(\text{mathcal{L}}_{\text{stage1}})$, 다음과 같이 정의됩니다.

(7)

여기서 t_k 는 정답 CAD 코드 토큰을 나타내고, $t_{<k}$ 는 이전 CAD 코드 토큰을 나타내며, $\text{prompt}_{\text{reverse}}$ 는 지시하는 텍스트 프롬프트를 나타냅니다.
이 모델은 B-rep 입력으로부터 CAD 코드를 재구성합니다. 이를 통해
이 과정에서 모델은 기하학적 정보에 내재된 설계 논리와 절차적 지식을 내면화합니다.

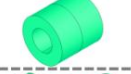
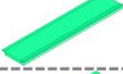
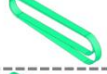
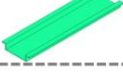
2단계: 하위 애플리케이션을 위한 멀티태스킹 적응. 두 번째
이 단계는 1단계에서 습득한 설계 논리 기능을 활용하여 다양한 문제를 해결합니다.
CAD 영역 내의 작업들. 사전 학습된 가중치를 초기화로 사용하여,
세 가지 하위 작업에 걸쳐 모델을 미세 조정합니다. (1) 완료: 모델
B-rep 입력과 함께 CAD 코드 시퀀스의 후반부를 수신합니다.
(2) 마스크 처리 후 전체 CAD 코드를 재구성합니다. 오류 수정: 주어진
명령 순서가 바뀌거나 매개변수가 잘못 구성된 CAD 코드가 있는 경우, 모델은 B-rep 형상과의 논리적 모순을 식별합니다.
수정된 코드를 출력합니다. (3) CAD-QA: B-rep과 해당 속성 또는 설계 의도에 관한 질문이 제시되면 모델은 기하학적 정보를 해석합니다.
주어진 정보를 바탕으로 네 가지 객관식 보기 중에서 정답을 선택하세요.

4가지 실험

4.1 실험 장치 구성

데이터 세트. 우리는 170K 시퀀스로 구성된 DeepCAD [35] 데이터 세트를 활용합니다.
우리는 DeepCAD의 원본 시퀀스를 파이프라인과 유사한 CAD 코드로 변환합니다.
형식을 지정하고 학습, 검증 및 테스트를 위해 90:5:5 비율로 분할합니다. 1단계의 경우

제목이 너무 길어서 생략되었습니다.

GT					
답캐드					
브렙코더					

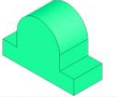
(a) 역설계

모델	빨간색으로 마스크 처리된 토큰을 사용하여 생성된 CAD 코드	
	<pre>sketch0 = [] loop0_0 = 루프() loop0_0.Line(endpoint=(70, 0)) loop0_0.Line(endpoint=(70, 60)) loop0_0.Arc(endpoint=(0, 60), degrees=180, counterclockwise=True) loop0_0.Line(endpoint=(0, 0)) loop0_1 = 루프() loop0_1.Line(endpoint=(60, 15)) loop0_1.Line(endpoint=(60, 30)) loop0_1.Line(endpoint=(10, 30))</pre>	<pre>loop0_1.Line(endpoint=(10, 15)) loop0_2 = 루프() loop0_2.Circle(center=(35, 60), radius=20) sketch0.add(loop0_0) sketch0.add(loop0_1) sketch0.add(loop0_2) Extrude(sketch=sketch0, orientation_degrees=(90, -90, 90),origin=(0, 0, 0), scale=0.75, extrude_distance=(10, 0), boolean_type='New', extent_type='One-sided')</pre>

(b) 완료

목표	잘못된 CAD	오류 수정 CAD
	<pre>..loop0_0.Arc(endpoint=(2, -1), degrees=104,counterclockwise=True) loop0_0.Line(endpoint=(93, -2)) loop0_0.Line(endpoint=(106, 13)) loop0_0.Line(endpoint=(3, 23)) loop0_0.Arc(endpoint=(93, 38), degrees=55, counterclockwise=True) loop0_0.Arc(endpoint=(-95, 0), degrees=66, counterclockwise=True)</pre>	<pre>..loop0_0.Arc(endpoint=(2, -2), degrees=90, counterclockwise=True) loop0_0.Line(endpoint=(93, -2)) loop0_0.Arc(endpoint=(95, 0), degrees=90, counterclockwise=True) loop0_0.Line(endpoint=(95, 21)) loop0_0.Arc(endpoint=(93, 23), degrees=90, counterclockwise=True) loop0_0.Line(endpoint=(2, 23))</pre>

(c) 오류 수정

모델	질문	답변	모델	질문	답변
	윗부분의 주된 모 양은 무엇인가요?	포인트LLM: 정사각형 BrepCoder: 아지		CAD 객체에는 몇 개의 절단부가 있습니까?	포인트LLM: 넷 BrepCoder: 둘

(d) CAD-QA

그림 4: 다양한 CAD 영역 작업에 대한 BrepCoder의 정성적 결과: (a) 역방향 (a) 엔지니어링, (b) 완료, (c) 오류 수정, (d) CAD 품질 보증.

정렬 및 1단계 역설계를 위해 B-rep 및 전체 CAD를 사용합니다.
코드 쌍. 2단계 데이터는 각 작업에 맞게 특별히 처리됩니다. 완료를 위해, 우리는 시퀀스의 처음 30%~50%를 유지하고 나머지는 마스크합니다. 오류 발생 시 정정하자면, 우리는 루프 내에서 명령 순서를 바꾸거나 매개변수에 노이즈를 주입합니다. 50%에서 80%의 비율을 적용합니다. CAD-QA의 경우 SGP-Bench [27] 데이터셋을 사용합니다. 1,000개의 모델과 객관식 문제가 80:10:10의 비율로 포함되어 있습니다. 측정 기준. 생성된 CAD 코드와 실제 결과 간의 일치도를 평가하기 위해 정답 시퀀스를 측정하기 위해 $\text{protect} \setminus \text{text} \{ \text{ACC} \}_\text{[cmd]}$ 와 $\text{protect} \setminus \text{text} \{ \text{ACC} \}_\text{[param]}$ 을 사용합니다. $\text{protect} \setminus \text{text} \{ \text{ACC} \}_\text{[cmd]}$ 는 다음을 측정합니다. 정확하게 예측된 명령의 비율. $\text{ACC} \setminus \text{[param]}$ 은 절대 오차가 허용 오차 δ 이내에 있는 매개변수의 비율을 계산합니다.

표 1: DeepCAD 데이터셋에 대한 역설계 성능 비교.
모따기 거리(CD)는 10^{-3} 으로 확대됩니다. 가장 좋은 결과는 굵게 표시되어 있습니다.

방법	ACCcmd ↑	ACCparam ↑	Med. CD ↓	IR ↓
DeepCAD [35]	85.95	74.22	10시 30분	12.08
MultiCAD [22]	89.43	79.81	8.09	11시 46분
View2CAD [46]	80.21	76.23	2.93	23.03
Brep2Seq [45]	-	-	33.5	11.87
CADCL [20]	90.31	81.52	0.972	13.98
BrepCoder 89.34		82.01	0.464	0.86

정확하게 예측된 명령만 해당됩니다.

(8)

(9)

여기서 N_c 는 전체 명령 수를 나타내고, K 는 총합을 나타냅니다.
매개변수 개수. $\text{protect } \mathbb{I}(\cdot)$ 는 0 또는 1을 출력하는 지표 함수입니다.
변수 c_i 와 $\text{protect } \hat{c}_i$ 는 각각 실제값과 예측값을 나타냅니다.
각각 i 번째 명령어입니다. 마지막으로, $p_{i,j}$ 와 $\text{protect } \hat{p}_{i,j}$ 는 정답을 나타냅니다.
 i 번째 명령의 j 번째 매개변수에 대한 예측값입니다.
3D 형상 품질을 평가하기 위해 무작위로 모따기 거리(CD)를 사용합니다.
8,096개의 데이터 포인트를 샘플링했습니다. 또한, 검증을 위해 무효 비율(IR)을 측정했습니다.
생성된 모델의 구조적 타당성.
구현 세부 사항. 우리는 Qwen 2.5-1.5B-Base [3,40]를 LLM 백본으로 사용합니다. 1단계 단일 모드 및 다중 모드 디코더는 각각 3개의 레이어로 구성됩니다.
그리고 UV-Net 인코더와 함께 처음부터 학습됩니다.
기준선. 우리는 BrepCoder를 각 항목별로 최첨단 기준선과 비교하여 평가합니다.
작업. 역공학을 위해 B-rep 입력에서 CAD 시퀀스를 생성하는 CADCL [20]을 선택합니다. 완료를 위해 CAD-Llama [19]와 비교합니다.
텍스트 주석과 초기 코드를 사용합니다. 오류 수정은 다음 기준에 따라 평가됩니다.
GPT[30] 및 Gemini[2, 8]를 포함한 상용 MLLM. CAD-QA의 경우, 우리는 ShapeLLM[26]과 같은 여러 포인트 클라우드 기반 MLLM을 훈련하고 비교합니다.
MiniGPT-3D [31] 및 PointLLM [39]은 동일한 조건에서 작동합니다.

4.2 역설계

이 섹션에서는 BrepCoder가 제공하는 리버스 엔지니어링 기능을 평가합니다.
1단계에서, 표 1은 정량적 평가 결과를 제시합니다. 첫째, 다음과 관련하여
기하학적 모양 유사성을 나타내는 모따기 거리 측정 기준에 따르면,
 $\times 10^{-3}$ 의 성능 개선을 달성 이는 모델당 약 52.26%의 성능 향상을 보여주며, 이전 최고 성능인 0.972 대비 0.464
했음을 의미합니다.
모델 CADCL [20]. 또한, 이 모델은 82.01%의 최고값을 기록했습니다.

제목이 너무 길어서 생략되었습니다.

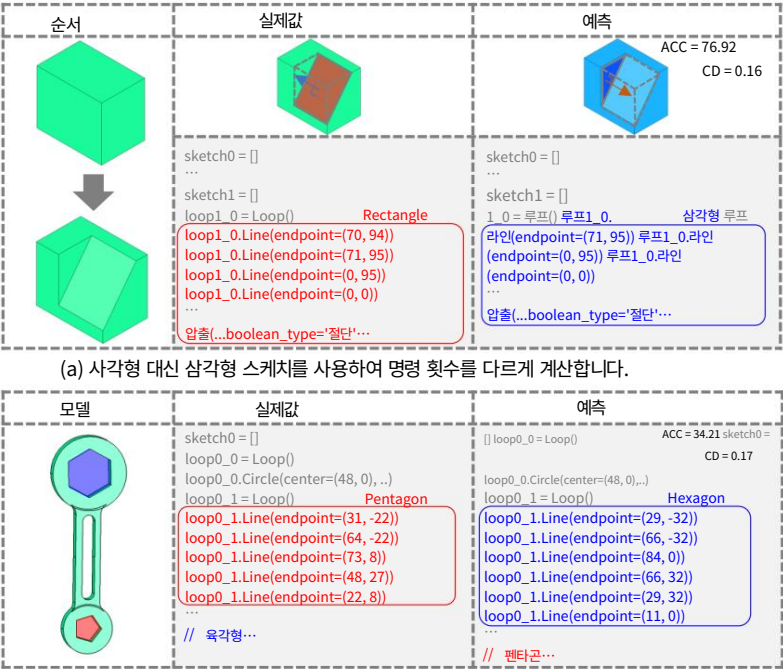


그림 5: 모델링 모호성 분석. 두 경우 모두, 명령 횟수와 순서가 다르더라도 모델은 기하학적으로 동일한 모양을 생성합니다.

ACC_{param} 지표에 대해 BrepCoder는 세밀한 기하학적 특징을 포착하고 이를 정확한 수치 매개변수로 변환하는 능력을 보여줍니다. 특히 주목할 만한 점은 무효 비율의 감소입니다. 기존 방법론은 높은 생성 실패율을 보이는 반면, BrepCoder는 0.86%의 값을 기록하며 대다수의 경우 실행 가능한 CAD 모델을 성공적으로 생성합니다. 이는 사전 학습된 LLM에 내재된 강력한 프로그래밍 언어 이해 및 추론 능력이 Python과 유사한 CAD 코드의 정확도를 보장하는 데 기여함을 시사합니다. 이를 보여주는 정성적 결과는 그림 4(a)에서 확인할 수 있습니다. 반대로 ACC_{cmd} 지표에 대해서는 다음과 같습니다.

BrepCoder는 CADCL이 달성한 90.31%에 비해 약간 낮은 89.34%의 성능을 보였습니다. 그러나 CAD 모델링 프로세스는 단일하지 않습니다. 동일한 3D 형상이라도 설계자의 의도나 방법론에 따라 다양한 순서로 정의될 수 있습니다[15,22,25]. 그림 5는 이러한 현상의 예를 보여줍니다. 그림 5(a)에 나타난 첫 번째 사례는 명령 개수의 불일치를 보여줍니다. 정답 시퀀스는 직육면체의 모서리를 깎는 절삭 작업을 수행하기 위해 4개의 명령으로 직사각형을 그립니다. 반면 BrepCoder는 기하학적으로 동일한 형상을 생성하기 위해 단 3개의 명령으로 삼각형을 그립니다. 이 시나리오에서 ACC_{cmd}는 정답 시퀀스에 비해 명령 개수가 다르기 때문에 낮게 측정됩니다.

두 번째 경우는 그림 5(b)에 나타난 바와 같이 명령 순서의 변형을 보여줍니다.

정답 데이터는 먼저 오각형을 생성하지만, BrepCoder는 그 반대의 경우를 생성합니다.

육각형이 먼저입니다. 명령 순서의 이러한 순열은 또한 더 낮은 값을 초래합니다.

 그림에도 불구하고 생성된 도형은 두 가지 모두에서 실제 형상과 일치합니다.

CD 값이 낮은 경우를 나타냅니다. 통계 분석은 $CD \leq 0.01$ 인 결과의 14.14%와 11.97%가 이러한 현상을 보이는 것을 보여줌으로써 이러한 모호성을 더욱 뒷받침합니다.

ACCcmd가 각각 90%와 80% 미만입니다. 따라서, 더 낮은 수치에도 불구하고

ACCcmd와 BrepCoder는 상당히 정확한 3D 형상을 재구성합니다.

4.3 다운스트림 작업

완료. 완료 작업은 초기 코드로부터 나머지 코드를 재구성합니다. CAD-Llama [19]를 포함한 기존 모델은 초기 코드를 받습니다.

가하학적 형태를 설명하는 텍스트 주석과 함께 제공됩니다. 예를 들어, 이 텍스트는 다음과 같습니다.

"다른 구성 요소들로 둘러싸인 중앙 원통..."과 같이 설명할 수 있습니다. 이와 대조적으로 BrepCoder는 B-rep 특징과 함께 초기 코드를 수신합니다. 표 2(a)에 제시된 바와 같이 BrepCoder는 92.69%의 `\protect \text {ACC} \_ {cmd}`와 `\protect \text {ACC} \_ {param}`을 달성합니다.

87.94라는 수치는 기존 CAD-Llama 모델보다 높은 성능을 보여줍니다. 이러한 성능 차이는 각 모델리티 간의 정보 풍부도 차이에서 비롯된 것으로 분석됩니다. 자연어 텍스트는

전체적인 가하학적 형태를 대략적으로 표현할 수 있지만, 정확한 수치 값과 위상 정보를 특징에 인코딩하는 데에는 한계가 있습니다.

B-rep 방식과 비교했을 때, BrepCoder는 논리적인 정보를 획득합니다.

3D 형상과 CAD 시퀀스 간의 사전 정렬을 통해

1단계에서 실시된 역공학 사전 교육. 결과적으로, 이것은

내면화된 사전 지식은 효과적인 수행에 직접적으로 기여합니다.

완료 작업에서 관찰되었습니다. 정성적 결과는 그림 4(b)에 제공됩니다.

오류 수정. 오류 수정 작업은 결함이 있는 CAD 코드를 수정합니다.

Gemini와 같이 B-rep 직접 입력을 처리할 수 없는 상용 MLLM

GPT를 사용하여 대각선 위쪽 시점에서 2D 이미지를 렌더링하고 제공했습니다.

고도 20°, 방위각 45°로 촬영했습니다. 한편, 기존 오류 수정 기준선인 ReCAD[6]는 지정된 최적 시점에서 이미지를 처리합니다.

또한, 이 연구는 OpenSCAD를 기반으로 자체 제작한 데이터셋을 활용합니다.

구문은 근본적으로 DeepCAD 데이터셋과 형식이 다릅니다.

표 2(b)의 실험 결과에서 알 수 있듯이 BrepCoder는 더 우수한 성능을 보입니다.

기본 모델. 특히 Gemini와 GPT의 경우, 이러한 모델은 다음과 같은 결과를 달성합니다.

높은 명령 정확도를 가지고 있지만, 낮은 매개변수 정확도로 인해 CD가 높아집니다.

이는 절대 좌표(예: 끝점=(140, 80)) 또는 정확한 치수(예: 반지름=48)와 같은 명시적인 공간 값을 추론하는 데 어려움을 강조합니다.

단일 2D 이미지. 또한, 서로 다른 데이터셋으로 인해 직접적인 비교가 어렵지만, 우리 모델은 ReCAD보다 낮은 CD 값을 달성합니다. 심지어

최적의 시점을 확보하기 위해 ReCAD는 2D 이미지의 고유한 한계에 직면합니다.

가려짐 현상과 정확한 수치 매개변수 추출을 방지합니다. 이와 대조적으로 Brep-Coder는 B-rep 입력을 처리하여

완전한 위상 구조와 정확한 가하학적 정보를 특징으로 효과적으로 인코딩합니다. 또한 BrepCoder는

3D 형상과 CAD 시퀀스 간의 논리적 정렬을 획득합니다.

표 2: 하위 작업 성능 비교: (a) 완료율, (b) 오류율
(c) 수정 및 CAD-QA.

메서드 ACCcmd ↑ ACCparam ↑			방법 ACCCAD-QA ↑	
GPT-4 [1]	51.18	38.96	ShapeLLM(7B) [26]	73%
GPT-3.5 [24]	42.98	36.29	MiniGPT-3D(2.7B) [31]	78%
LLaMA3 [32]	22.68	35.90	PointLLM(7B) [39]	81%
미스트랄 [14]	27시 30분	28.56	BrepCoder(1.5B)	79%
CAD-라마 [19]	73.87	57.14		
BrepCoder		87.94		

(a) 완료

방법 ACCcmd ↑ ACCparam ↑ Med. CD ↓ IR ↓			
제미니-2.5 [8]	71.53	42.30	350.30 22.53
GPT-5 [30]	82.83	43.41	399.78 25.84
ReCAD [6]	-	-	1.43 0.00
BrepCoder 99.08		93.74	0.335 0.32

(b) 오류 수정

(c) CAD-QA

1단계에서 실시된 역공학 사전 교육을 통해 사전에 파악했습니다.
결과적으로, 이러한 내재화된 사전 지식은 견고한 오류 수정 성능에 직접적으로 기여합니다. 정성적 결과는 그림 4(c)에 제시되어 있습니다.
CAD-QA. 마지막으로 CAD-QA 작업은 SGP-Bench [27] 데이터셋을 활용하여 CAD 모델에 대한 질문 답변 능력을 평가합니다. 표 2(c)의 실험 결과에서 볼 수 있듯이 BrepCoder는 다음과 같은 정확도를 달성했습니다.

79%라는 수치는, 이전에 달성했던 81%보다 불과 2%포인트 낮은 수치입니다.
PointLLM[39]에 의해. 이와 대조적으로 BrepCoder는 1.5B 매개변수의 소형 모델에 의존하며 코드 기반 역공학 작업에 대해서만 사전 학습됩니다.
이러한 조건에도 불구하고, 우리 모델은 매우 경쟁력 있는 성능을 달성합니다.
이러한 기능을 보여주는 정성적 결과는 그림 4(d)에 제시되어 있습니다.

4.4 절제술 연구

표 3(a)는 역설계 작업에 대한 제거 연구 결과를 나타냅니다.
1단계에서. 먼저, 1단계에서는 인코더를 처음부터 학습시켜 더 높은 성능을 달성합니다.
세그멘테이션 사전 학습 가중치로 초기화하는 것보다 성능이 더 좋습니다.
사전 학습된 인코더가 분할 지향적 특징에 편향되어 있기 때문입니다.
공간. 이와 대조적으로, 처음부터 학습하면 모델은 B-rep 형상과 CAD 코드 모두에 맞춰 정렬된 표현을 공동으로 학습할 수 있습니다. 둘째, 정렬
CoCa 프레임워크를 활용한 기능은 더 높은 성능을 보여줍니다.
CLIP [28] 방법과 비교했을 때. 이는 복잡한 CAD 코드 생성에서 캡션 손실의 중요한 역할을 확인시켜 줍니다. 셋째, B-rep 인코더를 고정합니다.
열리지 않은 상태로 두는 것보다 더 높은 성능을 나타냅니다. 이는 아마도 관절 때문일 것입니다.
인코더 학습 과정에서 사전 학습된 위상 정보가 저하됩니다.
정렬 과정. 결과적으로 이는 전반적인 일반화 능력을 저하시킵니다.
모델의 성능. 넷째, 1.5B와 0.5B 아키텍처를 비교하면 LLM 규모의 중요성을 확인할 수 있습니다. 성능 저하

표 3: (a) 역설계 및 (b) 후속 작업에 대한 제거 연구.

방법	ACCcmd ↑	ACCparam ↑	Med. CD ↓	IR ↓
사전 학습된 인코더 CLIP 인코더	89.39	81.93	0.499	0.92
Unfreeze	88.24	78.09	0.788	1.09
Qwen-2.5 0.5B 2레이어 1레이어 자체 개발	89.38	81.30	0.516	0.78
	88.51	79.77	0.612	1.01
	88.81	80.15	0.588	0.98
	89.13	80.66	0.565	0.92
	89.34	82.01	0.464	0.86

(a) 1단계: 역설계

과제	미터법	1단계 없음	
완성	ACCcmd ↑	92.53	92.96
	ACCparam ↑	87.91	87.94
오류 수정	ACCcmd ↑	99.43	99.08
	ACCparam ↑	93.53	93.74
	Med. CD ↓	0.405	0.335
	IR ↓	0.40	0.32
CAD-QA	ACC CAD-QA ↑	71%	79%

(b) 2단계: 하위 작업

0.5B 모델에서 관찰된 결과는 예측을 위한 모델의 역량이 불충분함을 의미합니다. 복잡한 CAD 코드. 마지막으로 프로젝트에 관해서는 3층 구성입니다. 1층 및 2층 구조 모두보다 더 높은 성능을 보입니다. 이것은 B-rep 특징을 매핑하는 데 있어 향상된 표현력 덕분입니다. LLM 텍스트 공간. 표 3(b)는 1단계 사전 학습이 후속 작업에 미치는 영향을 비교합니다. 완전히 학습된 모델은 전반적으로 다음과 같은 결과를 보여줍니다. 기준선(제외) 대비 모든 작업에서 성능 향상. 이 초기 단계는 '1단계 없음'으로 표시됩니다. 이는 모델이 효과적으로 작동함을 시사합니다. 3D 형상과 CAD 코드 간의 구조적 논리를 내재화합니다. 1단계 동안 발생하며, 이는 이후 하위 애플리케이션에 이점을 제공합니다. 1단계가 없는 경우 ACCcmd 지표에서 약간 더 높은 점수를 기록합니다. 오류 수정 작업에서 이러한 현상은 유사한 맥락에서 해석될 수 있습니다. 그림 5에서 앞서 논의된 모델링 모호성과 같이.

5. 결론

저희는 다양한 CAD 작업을 위해 B-rep 표현과 대규모 언어 모델을 통합한 멀티모달 프레임워크인 BrepCoder를 제안했습니다. 1단계에서는 2단계 전략은 역공학을 통해 의미론적 정렬을 확립합니다. 사전 훈련. 실험 결과는 이러한 기반이 완료, 오류 수정 등 후속 작업 전반에 걸쳐 강력한 성능을 가능하게 한다는 것을 검증합니다. CAD-QA. 향후 연구에서는 BrepCoder를 포인트 클라우드 입력으로 확장할 계획입니다. CAD-Recode[29]와 같은 역공학 작업을 원활하게 포괄합니다. CAD-SIGNet [16] 및 TransCAD [10]가 있습니다. BrepCoder가 발전할 것으로 예상합니다. 범용 CAD 에이전트 및 지능형 설계 자동화에 대한 연구.

참고 자료

1. Achiam, J., Adler, S., Agarwal, S., Ahmad, L., Akkaya, I., Aleman, FL 등:
GPT-4 기술 보고서. arXiv 사전 공개본 arXiv:2303.08774 (2023)
2. Anil, R., Borgeaud, S., Alayrac, JB, Yu, J., Soricut, R., Schalkwyk, J., et al.: Gem-ini: 매우 뛰어난 다중
모달 모델 제품군. arXiv 사전 공개 논문 arXiv:2312.11805 (2023)
3. Bai, J., Bai, S., Chu, Y., Cui, Z., Dang, K., Deng, X. 등: Qwen 기술 보고서. arXiv 사전 인쇄 arXiv:2309.16609
(2023)
4. Bank, D., Koenigstein, N., Giryas, R.: 오토인코더. In: 데이터 과학을 위한 기계 학습 핸드북: 데이터 마이닝 및
지식 발견 핸드북, pp.
353–374. 스프링거(2023)
5. Chen, C., Wei, J., Chen, T., Zhang, C., Yang, X., Zhang, S., Yang, B., Foo, CS, Lin, G., Huang, Q., Liu,
F.: CadCrafter: 제약 없는 이미지로부터 컴퓨터 자원 설계 모델 생성. In: CVPR. pp. 11073–11082 (2025)
6. Chen, J., Hei, X., Liu, H., Wei, Y., Deng, Z., Xie, J., Cai, Y., Qing, L.: CADReview: 오류 감지 및 수정을 통
한 CAD 프로그램 자동 검토. 제63회 전산언어학회 연례 학술대회 논문집(제1권: 장편 논문). 9909–9927쪽(2025
년 7월)
7. Chen, T., Yu, C., Hu, Y., Li, J., Xu, T., Cao, R., et al.: Img2cad: 구조화된 시각적 기하학을 이용한 단일 이미
지 기반 조건부 3D CAD 모델 생성. IEEE 산업 정보학 저널(2025)
8. Comanici, G., Bieber, E., Schaekermann, M., Pasupat, I., Sachdeva, N., Dhillon, I., et al.: Gemini
2.5: 고급 추론, 다중 모드, 장기 컨텍스트 및 차세대 에이전트 기능으로 한계를 뛰어넘다. arXiv 사전 공개 논문
arXiv:2507.06261 (2025)
9. Deng, L., Guo, H., Bai, Y., Dai, Y., Huang, H., Shi, Y.: BrepLLM: 대규모 언어 모델을 사용한 네이티브 경계 표
현 이해. arXiv preprint arXiv:2512.16413 (2025)
10. Dupont, E., Cherenkova, K., Mallis, D., Gusev, G., Kacem, A., Aouada, D.: Tran-sCAD: 포인트 클라우
드에서 CAD 시퀀스 추론을 위한 계층적 변환기.
출처: ECCV. 19–36쪽. Springer Nature Switzerland, Cham (2024년 9월)
11. Guan, Y., Wang, X., Xing, X., Zhang, J., Xu, D., Yu, Q.: CAD-Coder: 사고 연쇄 및 기하학적 보상을 이용한
텍스트-CAD 생성. arXiv 사전출판물 arXiv:2505.19713 (2025)
12. Heidari, N., Iosifidis, A.: 컴퓨터 자원 설계를 위한 기하학적 답러닝: A
실문조사. IEEE Access (2025)
13. Jayaraman, PK, Sanghi, A., Lambourne, JG, Willis, KDD, Davies, T., Shayani, H., Morris, N.: UV-Net:
경계 표현으로부터의 학습. In: CVPR. pp. 11703–11712 (2021)
14. Jiang, AQ, Sablayrolles, A., Mensch, A., Bamford, C., Chaplot, DS, de las Casas, D., Bressand, F.,
Lengyel, G., Lample, G., Saulnier, L., Lavaud, LR, Lachaux, MA, Stock, P., Le Scao, T., Lavril, T.,
Wang, T., Lacroix, T., 엘 사예드, W.: 미스트랄 7B. arXiv 사전 인쇄 arXiv:2310.06825 (2023)
15. Jung, M., Kim, M., Kim, J.: ContrastCAD: 컴퓨터 자원 설계 모델을 위한 대조 학습 기반 표현 학습. arXiv 사
전 출판물 arXiv:2404.01645 (2024)
16. Khan, MS, Dupont, E., Ali, SA, Cherenkova, K., Kacem, A., Aouada, D.: CAD-SIGNet: 레이어별 스케치
인스턴스 기반 어텐션을 이용한 포인트 클라우드로부터 CAD 언어 추론. In: CVPR. pp. 4713–4722 (2024)

16 M. Kim 외

17. Khan, MS, Sinha, S., Sheikh, TU, Stricker, D., Ali, SA, Afzal, MZ: Text2CAD: 초보자부터 전문가 수준까지의 텍스트 프롬프트를 사용하여 순차적인 CAD 디자인 생성. NeurIPS. vol. 37, pp. 7552–7579 (2024)
18. Lambourne, JG, Willis, KDD, Jayaraman, PK, Sanghi, A., Meltzer, P., Shayani, H.: B-repNet: 고체 모델을 위한 위상학적 메시지 전달 시스템.
에서: CVPR. 12773~12782페이지(2021)
19. Li, J., Ma, W., Li, X., Lou, Y., Zhou, G., Zhou, X.: CAD-Llama: 컴퓨터 지원 설계 파라메트릭 3D 모델 생성을 위한 대규모 언어 모델 활용. In: CVPR. pp. 18563–18573 (2025)
20. Liang, J., He, F., Fan, R., Chu, Y., Yan, X.: CADCL: 대조 학습을 통해 B-rep에서 매개변수 CAD 모델 재구성. 전산 설계 및 엔지니어링 저널 12(10), 176–184 (2025)
21. Liu, Y., Obukhov, A., Wegner, JD, Schindler, K.: Point2cad: 3D 포인트 클라우드에서 CAD 모델 역설계. In: CVPR. pp. 3763–3772 (2024)
22. Ma, W., Xu, M., Li, X., Zhou, X.: MultiCAD: 다중 모달 3D 컴퓨터 지원 설계 모델을 위한 대조적 표현 학습. In: 제32회 ACM 국제 정보 및 지식 관리 컨퍼런스 프로시딩. pp. 1766년~1776년 (2023년 10월)
23. Mews, M., Aynedinov, A., Schiller, V., Eisert, P., Akbik, A.: Don't Mesh with Me: 코드 생성 LLM의 미세 조정을 통해 메쉬 대신 구성적인 솔리드 기하학 생성. arXiv preprint arXiv:2411.15279 (2024)
24. Ouyang, L., Wu, J., Jiang, X., Almeida, D., Wainwright, C., Mishkin, P., et al.: 인간의 피드백을 통해 지시를 따르도록 언어 모델을 훈련시키기. In: 신경 정보 처리 시스템의 발전. vol. 35, pp. 27730–27744 (2022)
25. Para, W., Bhat, S., Guerrero, P., Kelly, T., Mitra, N., Guibas, L., Wonka, P.: Sketchgen: 제약 조건이 있는 CAD 스케치 생성. In: 신경 정보 처리 시스템의 발전. vol. 34, pp. 5077–5088 (2021)
26. Qi, Z., Dong, R., Zhang, S., Geng, H., Han, C., Ge, Z., Yi, L.: ShapeLLM: 신체화된 상호작용을 위한 범용 3D 객체 이해. 유럽 컴퓨터 비전 학회(ECCV) 회의록. 214–238쪽. Springer Nature Switzerland, Cham (2024년 9월)
27. Qiu, Z., Liu, W., Feng, H., Liu, Z., Xiao, TZ, Collins, KM, Tenenbaum, JB, Weller, A., Black, MJ, Schölkopf, B.: 대규모 언어 모델이 기호 그래픽 프로그램을 이해할 수 있을까요? arXiv 사전 인쇄본 arXiv:2408.08313 (2024)
28. Radford, A., Kim, JW, Hallacy, C., Ramesh, A., Goh, G., Agarwal, S., Sastry, G., Askell, A., Mishkin, P., Clark, J., Krueger, G., Sutskever, I.: 자연어 지도 학습을 통한 전이 가능한 시각 모델 학습. ICML. 8748–8763쪽 (2021년 7월)
29. Rukhovich, D., Dupont, E., Mallis, D., Cherenkova, K., Kacem, A., Aouada, D.: CAD-Recode: 포인트 클라우드의 라버스 엔지니어링 CAD 코드. 에서: ICCV. 9801~9811페이지(2025)
30. Singh, A., Fry, A., Perelman, A., Tart, A., Ganesh, A., El-Kishky, A., et al.: OpenAI GPT-5 시스템 카드. arXiv 사전 공개 논문 arXiv:2601.03267 (2026)
31. Tang, Y., Han, X., Li, X., Yu, Q., Hao, Y., Hu, L., Chen, M.: MiniGPT-3D: 2D 사전 정보를 사용하여 대규모 언어 모델과 3D 포인트 클라우드를 효율적으로 정렬하는 방법. 제32회 ACM 국제 멀티미디어 학회 회의록. 6617–6626쪽 (2024년 10월)
32. Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, MA, Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., Lample, G.: LLaMA: 개방적이고 효율적인 기초 언어 모델. arXiv 사전 인쇄 arXiv:2302.13971 (2023)

33. Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, AN, Kaiser, , Polosukhin, I.: 주의가 필요한 전부입니다. 에서: NeurIPS. 권. 30 (2017)
34. Wang, S., Chen, C., Le, X., Xu, Q., Xu, L., Zhang, Y., Yang, J.: CAD-GPT: 공간 추론 기능이 강화된 다중 모달 LLM을 이용한 CAD 건설 시퀀스 합성. AAAI. vol. 39, pp. 7880–7888 (2025년 4월)
35. Wu, R., Xiao, C., Zheng, C.: DeepCAD: 컴퓨터 자원 설계 모델을 위한 심층 생성 네트워크. ICCV. pp. 6772–6782 (2021)
36. Wu, T., Yang, G., Li, Z., Zhang, K., Liu, Z., Guibas, L., Lin, D., Wetzstein, G.: GPT-4V(ision)은 텍스트를 3D로 변환하는 인간 중심 평가 도구입니다. CVPR, 22227–22238쪽(2024)
37. Wu, Z., Pan, S., Chen, F., Long, G., Zhang, C., Yu, PS: 그래프 신경망에 대한 종합적인 조사. IEEE Transactions on Neural Networks and Learning Systems 32(1), 4–24 (2020)
38. Xu, J., Wang, C., Zhao, Z., Liu, W., Ma, Y., Gao, S.: CAD-MLLM: MLLM을 이용한 다중 모달리티 조건부 CAD 생성의 통합. arXiv 사전출판물 arXiv:2411.04954 (2024)
39. Xu, R., Wang, X., Wang, T., Chen, Y., Pang, J., Lin, D.: PointLLM: 포인트 클라우드를 이해하기 위한 대규모 언어 모델의 역량 강화. 유럽 컴퓨터 비전 학회(ECCV) 회의록. pp. 131–147. Springer Nature Switzerland, Cham (2024년 9월)
40. Yang, A., Yang, B., Hui, B., Zheng, B., Yu, B., Zhou, C. 등: Qwen2 기술 보고서. arXiv 사전 공개본 arXiv:2407.10671 (2024)
41. Yin, S., Fu, C., Zhao, S., Li, K., Sun, X., Xu, T., Chen, E.: 멀티모달 대규모 언어 모델에 대한 조사. National Science Review 11(12), nwae403 (2024)
42. Yu, J., Wang, Z., Vasudevan, V., Yeung, L., Seyedhosseini, M., Wu, Y.: CoCa: 대조 캡셔너는 이미지-텍스트 기반 모델입니다. arXiv 사전 출판물 arXiv:2205.01917 (2022)
43. Yuan, H., Xu, J., Pan, H., Bousseau, A., Mitra, NJ, Li, C.: CADTalk: CAD 프로그램의 의미론적 주석을 위한 알고리즘 및 벤치마크. In: CVPR. pp. 3753–3762 (2024)
44. Zhang, L., Le, B., Akhtar, N., Lam, SK, Ngo, D.: 컴퓨터 자원 설계를 위한 대규모 언어 모델: 설문 조사. ACM Computing Surveys (2025)
45. Zhang, S., Guan, Z., Jiang, H., Ning, T., Wang, X., Tan, P.: Brep2Seq: 컴퓨터 자원 설계 모델의 재구성 및 생성을 위한 데이터셋 및 계층적 디러닝 네트워크. 계산 설계 및 엔지니어링 저널 11(1), 110–134 (2024)
46. Zhang, Y., He, F., Fan, R., Fan, B.: View2cad: 다중 뷰를 CAD 명령 시퀀스로 파싱. In: 2024년 제27회 컴퓨터 자원 협업 설계 국제 컨퍼런스(CSCWD) 프로시딩. pp. 2949–2954. IEEE (2024년 5월)
47. Zhao, WX, Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., et al.: 대규모 조사 언어 모델. arXiv 사전 출판물 arXiv:2303.18223 (2023)
48. Zhou, S., Tang, T., Zhou, B.: CADParser: B-Rep CAD를 위한 시퀀스 모델링 학습 접근법. IJCAI. pp. 1804–1812 (2023년 8월)